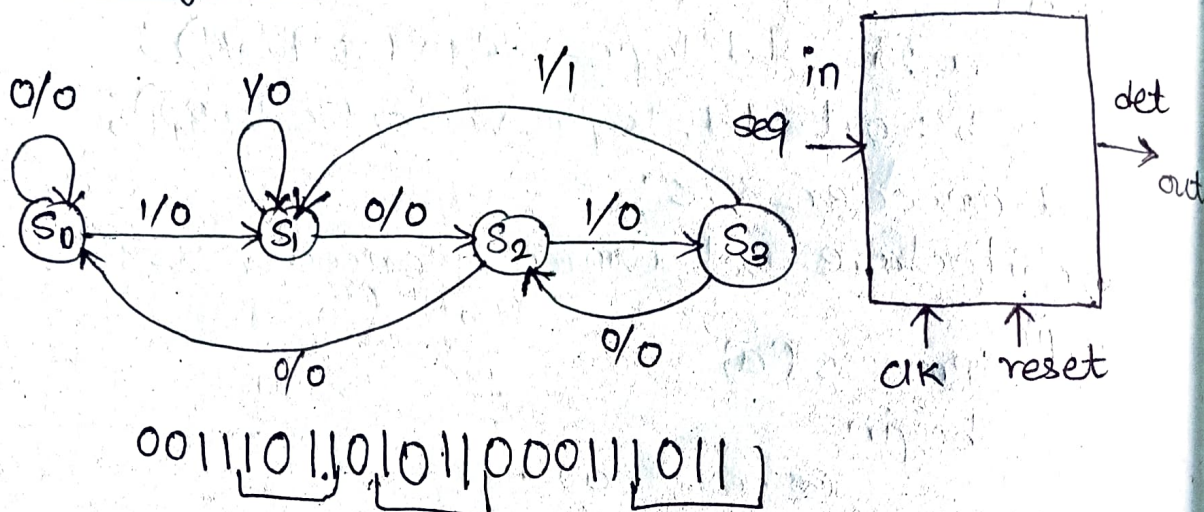


⑤ VHDL code for sequence detector which detects "1011" with overlapping.

☞ To detect sequence "1011".

i) state diagram:



ii) state Table:

Present state	Next state		Output, Y	
	X=0	X=1	X=0	X=1
S ₀	S ₀	S ₁	0	0
S ₁	S ₂	S ₁	0	0
S ₂	S ₀	S ₃	0	0
S ₃	S ₂	S ₁	0	1

Let $S_0 = 00$; $S_1 = 01$; $S_2 = 10$; $S_3 = 11$

Present state	Next state		Output, Y	
	X=0	X=1	X=0	X=1
00	00	01	0	0
01	10	01	0	0
10	00	11	0	0
11	10	01	0	1

VHDL code:

```
library ieee;  
use ieee.std-logic-1164.all;  
entity seq-det is  
  port(  
    seq, clk, reset; in std-logic;  
    det: out std-logic);
```

```
end seq-det;  
architecture Behavioral of seq-det is
```

```
  type state_type is (s0, s1, s2, s3);
```

```
  signal state: state_type := s0;
```

```
begin
```

```
  process (clk, reset)
```

```
  begin
```

```
    if (reset = '1') then
```

```
      det <= '0';
```

```
      state <= s0;
```

```
    elsif (rising_edge(clk)) then
```

```
      case bit is  
      case state is
```

```
        when s0 =>
```

```
          det <= '0';
```

```
          if (seq = '0') then
```

```
            state <= s0;
```

```
          else
```

```
            state <= s1;
```

```
          end if;
```

----- when s1 => -----

~~det <= '0';~~

if (seq = '0') then

state <= s2;

else

state <= s1;

endif;

----- when s2 => -----

if (seq = '0') then

state <= s0;

else

state <= s3;

endif;

----- when s3 => -----

if (seq = '0') then

state <= s2;

else

state <= s1;

~~endif;~~

det <= '1';

~~endif;~~
endif;

----- when others => -----

NULL;

end case;

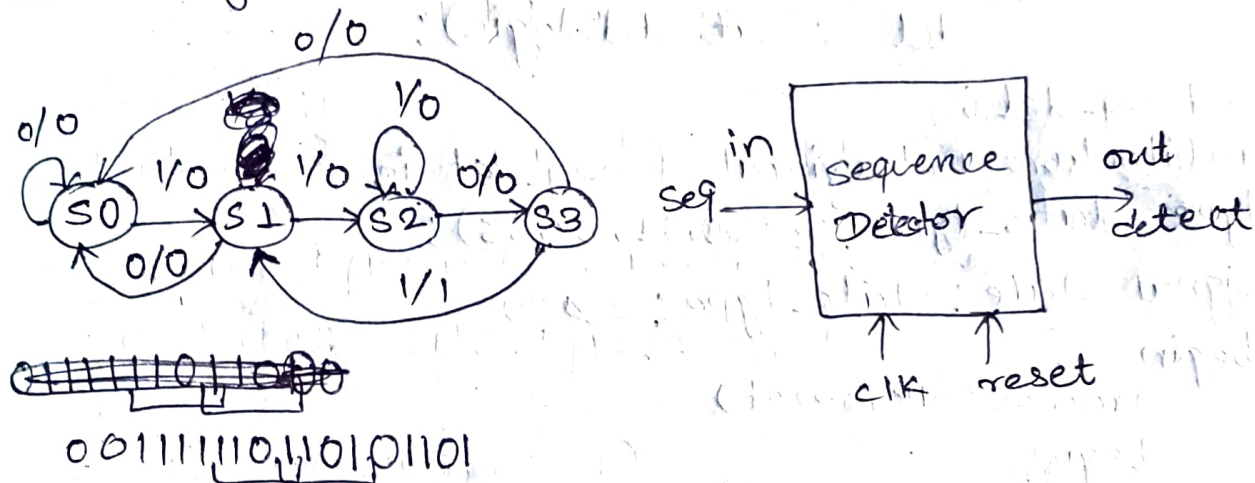
end if;

end process;

end Behavioral;

6.6) Write VHDL code which results an output '1' when a sequence "1101" is detected else an output results a '0' considering overlapping.

i) state diagram:



ii) state table:

Present state	Next state		Output, Y	
	X=0	X=1	X=0	X=1
S0	S0	S1	0	0
S1	S0	S2	0	0
S2	S3	S2	0	0
S3	S0	S1	0	1

Let S0 = 00, S1 = 01, S2 = 10, S3 = 11

Present State	Next state		Output, Y	
	X=0	X=1	X=0	X=1
00	00	01	0	0
01	00	10	0	0
11	11	10	0	0
10	00	01	0	1

*VHDL code:

library ieee;

use ieee.std_logic_1164.all;

entity seq-det is

port (

seq, clk, reset : in std_logic;

det : out std_logic);

end seq-det;

architecture Behavioral of seq-det is

type state_type is (s0, s1, s2, s3);

signal state : state_type := s0

begin

process (clk, reset)

begin

if (reset = '1') then

det <= '0';

state <= s0;

elsif (rising_edge(clk)) then

case state is

----- when s0 => -----

~~if (seq = '0')~~
det <= 0;

if (seq = '0') then

state <= s0;

else

state <= s1;

end if;

when s1 =>

if (seq = '0') then

state <= s0;

else

state <= s2;

end if;

when s2 =>

if (seq = '0') then

state <= s3;

else

state <= s2;

end if;

when s3 =>

if (seq = '0') then

state <= s0;

else

state <= s1;

det <= 1;

end if;

when others =>

NULL;

end case;

end if;

end process;

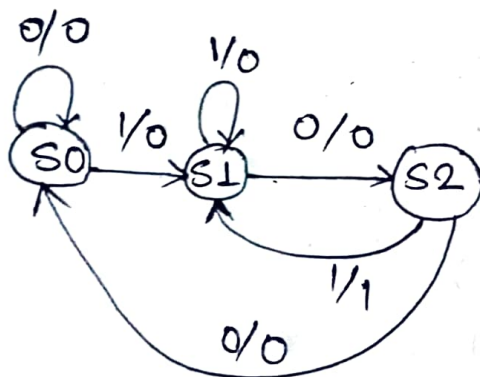
end Behavioral;

6.b) Write a VHDL program for which output will be 1 when the sequence 101 is detected considering overlapping.

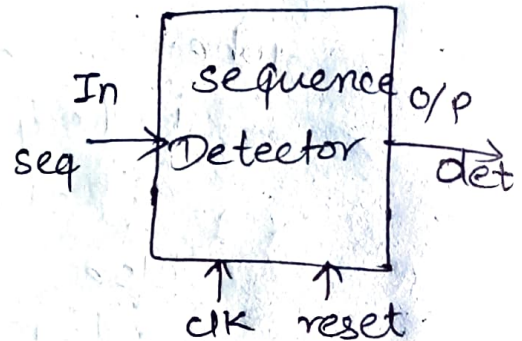
seq

101

i) state diagram:



001110101001101



ii) state table:

Present state	Next state		Output, Y	
	X=0	X=1	X=0	X=1
S0	S0	S1	0	0
S1	S2	S1	0	0
S2	S0	S1	0	1

VHDL code:

```
library ieee;
```

```
use ieee.std_logic-1164.all;
```

```
entity seq-det is
```

```
port (
```

```
    seq, clk, reset: in std_logic;
```

```
    det: out std_logic);
```

```
end seq-det;
```

architecture Behavioral of seq-det is

```
type state-type is (s0, s1, s2);
```

```
signal state: state-type := s0;
```

```
begin
```

```
    process (clk, reset)
```

```
    begin
```

```
        if (reset = '1') then
```

```
            det <= '0';
```

```
            state <= s0;
```

```
        elsif (rising-edge(clk)) then
```

```
            case state is
```

```
                when s0 =>
```

```
                    det <= '0';
```

```
                    if (seq = '0') then
```

```
                        state <= s0;
```

```
                    else
```

```
                        state <= s1;
```

```
                    end if;
```


when s1 =>

if (seq = '0') then

state <= s2;

else

state <= s1;

end if;

when s2 =>

if (seq = '0') then

state <= s0;

else

state <= s1;

end if;

when others =>

NULL;

end case;

end if;

end process;

end Behavioral;

② VHDL for full adder:

library ieee;

use ieee.std_logic_1164.all;

entity FA is

port (

A, B, Cin : in std_logic;

S, Cout : out std_logic);

end FA;

architecture structural of FA is

component HA

Port (

A, B; in std_logic;

S, C; out std_logic);

end component;

component OR_gate

Port (A, B: in std_logic;

Y: out std_logic);

end component

signal S1, C1, C2 : std_logic

begin

HA1: HA Port map (A $\Rightarrow$ A, B $\Rightarrow$ B, S $\Rightarrow$ S1, C $\Rightarrow$ C1);

HA2: HA Port map (A $\Rightarrow$ S1, B $\Rightarrow$ Cin, S $\Rightarrow$ S, C $\Rightarrow$ C2);

OR1: OR_gate Port map (A $\Rightarrow$ C1, B $\Rightarrow$ C2, Y $\Rightarrow$ Cout);

end structural;

④ VHDL code for 4-bit full adder:

library ieee;

use ieee.std_logic-1164.all;

entity 4Bit_FA is

Port (

A, B : in std_logic_vector (3 downto 0);

Cin : in std_logic;

S : out std_logic_vector (3 downto 0);

Cout : out std_logic);

end 4Bit_FA;

architecture structural of 4Bit_FA is

signal c : std_logic_vector (2 downto 0);

~~Port (~~

~~A, B, Cin : in std~~

Component FA .

Port (

A, B, Cin : in std_logic;

S, Cout : out std_logic);

end component;

begin

FA0: FA Port map (A(0), B(0), Cin, S(0), C(0));

FA1: FA Port map (A(1), B(1), C(0), S(1), C(1));

FA2: FA Port map (A(2), B(2), C(1), S(2), C(2));

FA3: FA Port map (A(3), B(3), C(2), S(3), Cout);

end structural;

⑧ VHDL code for 8x1 MUX:

sof

library ieee;

use ieee.std-logic-1164.all;

entity MUX-8-1 is

Port (

D: in std-logic-vector (7 downto 0);

S: in std-logic-vector (2 downto 0);

Y: out std-logic);

end MUX-8-1;

architecture structural of MUX-8-1 is

component MUX-2-1

Port (

A, B: in std-logic;

sel: in std-logic;

Y: out std-logic);

end component;

signal Y: std-logic-vector (5 downto 0);

begin

U1:

MUX-2-1 Port map (A $\Rightarrow$ D(0), B $\Rightarrow$ D(1),

sel $\Rightarrow$ S(0), Y $\Rightarrow$ Y(1));

U2: MUX-2-1 Port map (A $\Rightarrow$ D(2), B $\Rightarrow$ D(3),

sel $\Rightarrow$ S(0), Y $\Rightarrow$ Y(2));

U3: MUX-2-1 Port map (A $\Rightarrow$ D(4), B $\Rightarrow$ D(5),

sel $\Rightarrow$ S(0), Y $\Rightarrow$ Y(3));

U4: MUX-2-1 Port map (A $\Rightarrow$ D(6), B $\Rightarrow$ D(7),

sel $\Rightarrow$ S(0), Y $\Rightarrow$ Y(4));

U5: MUX-2-1 Port map (A $\Rightarrow$ Y(1), B $\Rightarrow$ Y(2), sel = s(1),
Y $\Rightarrow$ Y(5));

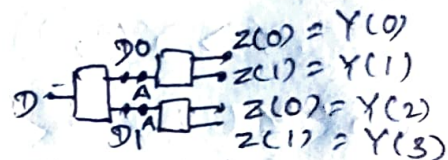
U6: MUX-2-1 Port map (A $\Rightarrow$ Y(3), B $\Rightarrow$ Y(4), sel =
sel $\Rightarrow$ s(0), Y $\Rightarrow$ Y(6));

U7: MUX-2-1 Port map (A $\Rightarrow$ Y(5), B $\Rightarrow$ Y(6),
sel $\Rightarrow$ s(2), Y $\Rightarrow$ Y);

end structural;

7) VHDL code for 1x4 Demux:

sg



```
library ieee;
use ieee.std-logic-1164.all;
entity Demux-1-4 is
    Port (
        D: in std-logic;
        S: in std-logic-vector(1 downto 0);
        Y: out std-logic-vector(3 downto 0));
```

```
end Demux-1-4;
```

```
architecture structural of Demux-1-4 is
    component Demux-1-2
```

```
        Port (
            A: in std-logic;
            sel: in std-logic;
            Z: out std-logic-vector(1 downto 0));
```

```
end component;
```

```
signal D0, D1: std-logic;
begin
```

```
U1: Demux-1-2 Port map (A => D, sel = S(1),
    Z(0) => D0, Z(1) => D1);
```

```
U2: Demux-1-2 Port map (A => D0, sel = S(0),
    Z(0) => Y(0), Z(1) => Y(1));
```

```
U3: Demux-1-2 Port map (A => D1, sel = S(0),
    Z(0) => Y(2), Z(1) => Y(3));
```

```
end structural;
```


⑧ VHDL code for 1x8 Demux:

```
library ieee;  
use ieee.std_logic-1164.all;  
entity Demux-1-8 is  
    Port (  
        D: in std_logic;  
        S: in std_logic_vector (2 downto 0);  
        Y: out std_logic_vector (7 downto 0));
```

end Demux-1-8;

architecture structural of Demux-1-8 is

component Demux-1-2

Port (

A: in std_logic;

sel: in std_logic;

z0, z1: out std_logic);

end component;

signal D0, D1: std_logic;

signal D2, D3, D4, D5, ~~D6~~: std_logic;

~~begin~~
begin

U1: Demux-1-2 Port map (A $\Rightarrow$ D, sel $\Rightarrow$ S(2),
z0 $\Rightarrow$ D0, z1 $\Rightarrow$ D1);

U2: Demux-1-2 Port map (A $\Rightarrow$ D0, sel $\Rightarrow$ S(1),
z0 $\Rightarrow$ D2, z1 $\Rightarrow$ D3);

U3: Demux-1-2 Port map (A $\Rightarrow$ D1, sel $\Rightarrow$ S(1),
z0 $\Rightarrow$ D4, z1 $\Rightarrow$ D5);

U4: Demux-1-2 Port map (A $\Rightarrow$ D2, sel $\Rightarrow$ S(0),
z0 $\Rightarrow$ Y(0), z1 $\Rightarrow$ Y(1));

U5: Demux-1-2 Port map (A $\Rightarrow$ D3, sel $\Rightarrow$ S(0),
z0 $\Rightarrow$ Y(2), z1 $\Rightarrow$ Y(3));

U6: Demux-1-2 Port map (A $\Rightarrow$ D4, sel $\Rightarrow$ S(0),
z0 $\Rightarrow$ Y(4), z1 $\Rightarrow$ Y(5));

U7: Demux-1-2 Port map (A $\Rightarrow$ D5, sel $\Rightarrow$ S(0),
z0 $\Rightarrow$ Y(6), z1 $\Rightarrow$ Y(7));

end structural;

④ VHDL code for 4x1 MUX:

Python, NAST



library ieee;

use ieee.std-logic-1164.all;

entity MUX_4-1 is

Port (

D: in std-logic-vector (3 downto 0);

S: in std-logic-vector (1 downto 0);

Y: out std-logic);

end MUX_4-1;

architecture structural of MUX_4-1 is

component MUX_2-1

Port (

A, B: in std-logic;

sel: in std-logic;

Y: out std-logic);

end ~~component~~ component;

signal Y1, Y2: std-logic;

begin

U1: MUX_2-1 Port map (A => D(0), B => D(1),
sel => S(0), Y => Y1);

U2: MUX_2-1 Port map (A => D(2), B => D(3),
sel => S(0), Y => Y2);

U3: MUX_2-1 Port map (A => Y1, B => Y2,
sel => S(1), Y => Y);

end structural;